



SERVICIO NACIONAL DE APRENDIZAJE – SENA

Programa de formación: Análisis y Desarrollo de Software

EVIDENCIA DE DESEMPEÑO

Diseño y Desarrollo de servicios web - proyecto GA7-220501096-AA5-EV03

Proyecto formativo

SIGTAL – Sistema Integral de Gestión para el Taller Automotriz Londoño

Pruebas de la API mediante Postman

Aprendiz: Mauricio José Polo Acosta

2 de octubre de 2026



1. Introducción

El presente documento contiene la evidencia de las pruebas realizadas sobre la API REST del proyecto SIGTAL – Sistema Integral de Gestión para Taller Automotriz Electricista Londoño. Las pruebas se ejecutaron utilizando Postman para verificar los servicios web implementados, sus respuestas HTTP y el manejo de escenarios exitosos y de error.

La API fue desarrollada con Java, Spring Boot, Spring Data JPA, Maven y MySQL. Para esta evidencia se documentan únicamente los módulos actualmente implementados en la API: Autenticación, Clientes y Vehículos e Inventario.

2. Objetivos

2.1 Objetivo general

Verificar mediante Postman el funcionamiento de los servicios REST implementados para la API SIGTAL, validando respuestas exitosas, validaciones y manejo de recursos inexistentes.

2.2 Objetivos específicos

- Comprobar el funcionamiento del servicio de autenticación.
- Verificar las operaciones disponibles para Clientes y Vehículos.
- Validar las operaciones CRUD implementadas para Inventario.
- Comprobar el manejo de recursos inexistentes mediante HTTP 404.
- Comprobar la validación de códigos de producto duplicados.
- Registrar los resultados obtenidos durante las pruebas.

3. Alcance de las pruebas

Módulo	Estado	Alcance
Autenticación	Implementado y probado	Inicio de sesión mediante API REST.
Clientes y Vehículos	Implementado y probado	Operaciones disponibles del controlador.
Inventario	Implementado y probado	Consultar, listar, registrar, actualizar y eliminar productos.
Configuración	Fuera del alcance	No existen endpoints en esta API.

4. Herramientas y entorno de prueba

- Postman: herramienta utilizada para enviar solicitudes HTTP y verificar respuestas.
- Spring Boot 3.5.6: framework de la API.
- Java: lenguaje de desarrollo.
- Maven: compilación y empaquetado.
- MySQL: base de datos.

- Servidor local: <http://localhost:8080>.

5. Procedimiento de pruebas

1. Verificar que Spring Boot estuviera ejecutándose en el puerto 8080.
2. Abrir Postman y seleccionar el método HTTP.
3. Introducir la URL del endpoint.
4. Configurar parámetros o JSON cuando correspondiera.
5. Enviar la solicitud.
6. Verificar código HTTP y contenido de la respuesta.
7. Registrar y comparar el resultado.

6. Matriz de pruebas

ID	Módulo	Prueba	Estado
01	01 - Autenticación	01 - Login exitoso	Ejecutada / incluida en colección
02	01 - Autenticación	01 - Usuario inexistente	Ejecutada / incluida en colección
03	01 - Autenticación	02 - Contraseña incorrecta	Ejecutada / incluida en colección
04	02 - Clientes y Vehículo	01 - Listar clientes	Ejecutada / incluida en colección
05	02 - Clientes y Vehículo	02-Consultar cliente por ID	Ejecutada / incluida en colección
06	02 - Clientes y Vehículo	03 - Vehículos por cliente	Ejecutada / incluida en colección
07	02 - Clientes y Vehículo	04 - Registrar cliente y vehículo	Ejecutada / incluida en colección
08	02 - Clientes y Vehículo	05-Documento duplicado	Ejecutada / incluida en colección
09	02 - Clientes y Vehículo	06 - Placa duplicada	Ejecutada / incluida en colección
10	02 - Clientes y Vehículo	07 - Actualizar cliente	Ejecutada / incluida en colección
11	02 - Clientes y Vehículo	08 - Actualizar vehículo	Ejecutada / incluida en colección
12	02 - Clientes y Vehículo	09 - Eliminar vehículo	Ejecutada / incluida en colección
13	02 - Clientes y Vehículo	10 - Verificar eliminación del vehículo	Ejecutada / incluida en colección
14	02 - Clientes y Vehículo	11 - Eliminar cliente	Ejecutada / incluida en colección
15	02 - Clientes y Vehículo	12 - Consultar cliente eliminado	Ejecutada / incluida en colección
16	03 - Inventario	01 - Listar productos	Ejecutada / incluida en colección
17	03 - Inventario	02 - Registrar producto	Ejecutada / incluida en colección



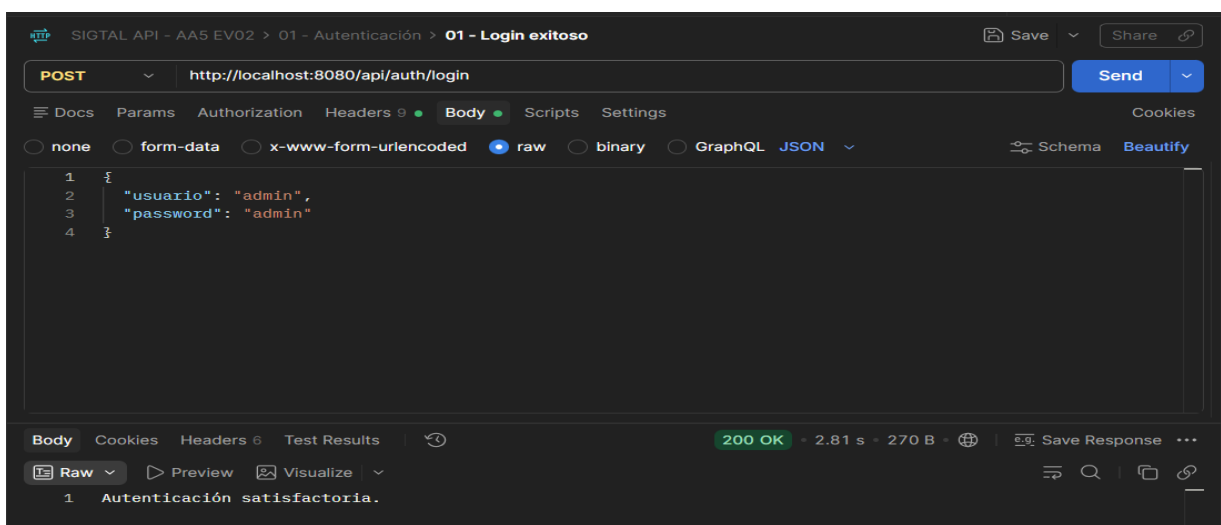
18	03 - Inventario	GET 03 - Consultar producto por id	Ejecutada / incluida en colección
19	03 - Inventario	04 - Actualizar producto	Ejecutada / incluida en colección
20	03 - Inventario	05 - Eliminar producto	Ejecutada / incluida en colección
21	03 - Inventario	06 - Verificar producto eliminado	Ejecutada / incluida en colección
22	03 - Inventario	07 - Código de producto duplicado	Ejecutada / incluida en colección
23	03 - Inventario	08 - Actualizar producto	Ejecutada / incluida en colección
24	03 - Inventario	09 - Producto inexistente	Ejecutada / incluida en colección
25	03 - Inventario	10 - Eliminar el producto ID 9	Ejecutada / incluida en colección
26	03 - Inventario	- Comprobación de producto eliminado	Ejecutada / incluida en colección
27	03 - Inventario	12 - ID inexistente	Ejecutada / incluida en colección

Evidencias de resultados en PostMan

Pruebas Autenticación

Prueba 1 - Login exitoso

Se realizó la prueba 01 - Login exitoso, verificando el funcionamiento del servicio de autenticación con credenciales válidas. El resultado permite comprobar que el proceso de inicio de sesión funciona correctamente.





Prueba 2 - Usuario inexistente

Se realizó la prueba 02 - Usuario inexistente, con el propósito de comprobar el comportamiento de la API cuando se intenta iniciar sesión con un usuario que no se encuentra registrado.

The screenshot shows a REST client interface for a test named "02 - Usuario inexistente". The request is a POST to `http://localhost:8080/api/auth/login` with a JSON body: `{ "usuario": "usuario_inexistente", "password": "admin" }`. The response is a `401 Unauthorized` status with a message: `Usuario no encontrado.`

```
1 {
2   "usuario": "usuario_inexistente",
3   "password": "admin"
4 }
```

Body Cookies Headers 6 Test Results 401 Unauthorized • 81 ms • 263 B • Save Response

Raw Preview Pass the correct auth credentials

```
1 Usuario no encontrado.
```

Prueba 3 - Contraseña incorrecta

Se realizó la prueba 03 - Contraseña incorrecta, verificando que el sistema valide las credenciales y rechace el acceso cuando la contraseña suministrada no corresponde al usuario.

The screenshot shows a REST client interface for a test named "03 - Contraseña incorrecta". The request is a POST to `http://localhost:8080/api/auth/login` with a JSON body: `{ "usuario": "admin", "password": "incorrecta" }`. The response is a `401 Unauthorized` status with a message: `Contraseña incorrecta.`

```
1 {
2   "usuario": "admin",
3   "password": "incorrecta"
4 }
```

Body Cookies Headers 6 Test Results 401 Unauthorized • 41 ms • 264 B • Save Response

Raw Preview Pass the correct auth credentials

```
1 Contraseña incorrecta.
```



Pruebas de Clientes y Vehículo

Prueba 1 - Listar clientes

Se realizó la prueba 01 - Listar clientes, verificando que la API permita consultar y mostrar los clientes registrados en el sistema.

GET `http://localhost:8080/api/clientes-vehiculos` **Send**

Docs Params Authorization Headers 6 **Body** Scripts Settings Cookies

Body Cookies Headers 6 Test Results **200 OK** • 303 ms • 721 B • Save Response

```
{
  "idCliente": 1,
  "nombre": "Carlos Andrés Martínez",
  "documento": "1098765432",
  "telefono": "3015282378",
  "correo": "carlos.martinez@gmail.com",
  "direccion": "Carrera 45 # 72-30"
},
{
  "idCliente": 31,
  "nombre": "Juan Pérez Acosta",
  "documento": "12345678",
  "telefono": "3004567891",
  "correo": "juanperez@gmail.com",
  "direccion": "Carrera 45 # 72-30"
},
{
  "idCliente": 38,
  "nombre": "Pedro Gómez",
  "documento": "7000000001",
  "telefono": "3007000001",
  "correo": "pedro.gomez@gmail.com",
  "direccion": "Barranquilla"
}
```

Prueba 2 - Consultar cliente por ID

Se realizó la prueba 02 - Consultar cliente por ID, comprobando que el sistema permita obtener la información de un cliente específico mediante su identificador.

GET `http://localhost:8080/api/clientes-vehiculos/1` **Send**

Docs **Params** Authorization Headers 6 Body Scripts Settings Cookies

Key	Value	Description	Bulk Edit
Key	Value	Description	

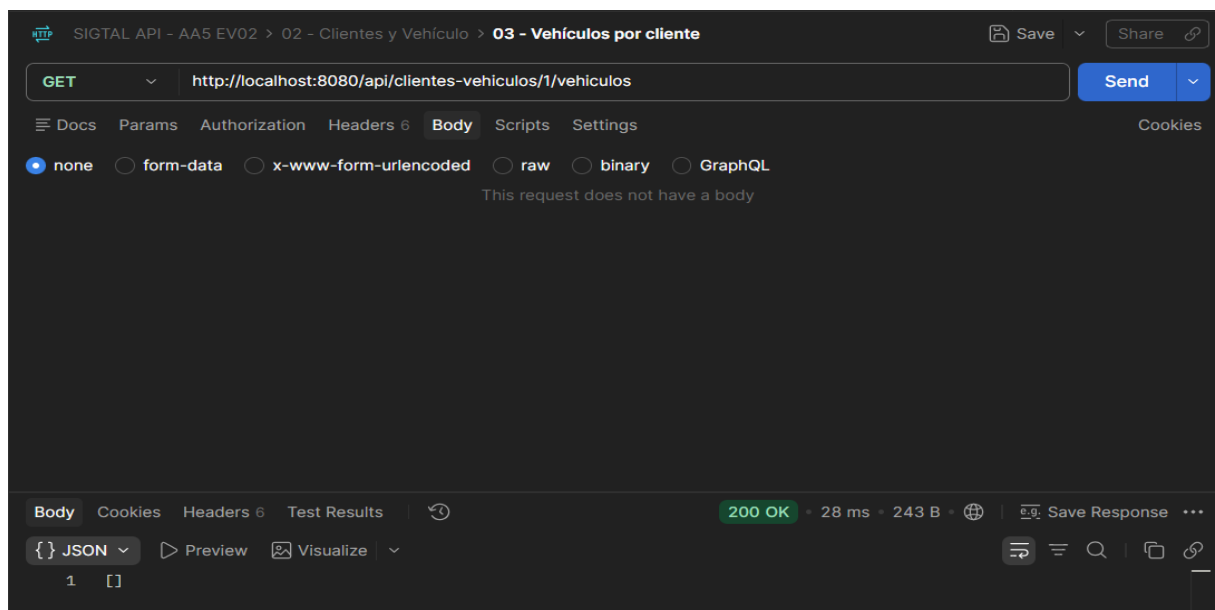
Body Cookies Headers 6 Test Results **200 OK** • 103 ms • 411 B • Save Response

```
{
  "idCliente": 1,
  "nombre": "Carlos Andrés Martínez",
  "documento": "1098765432",
  "telefono": "3015282378",
  "correo": "carlos.martinez@gmail.com",
  "direccion": "Carrera 45 # 72-30"
}
```



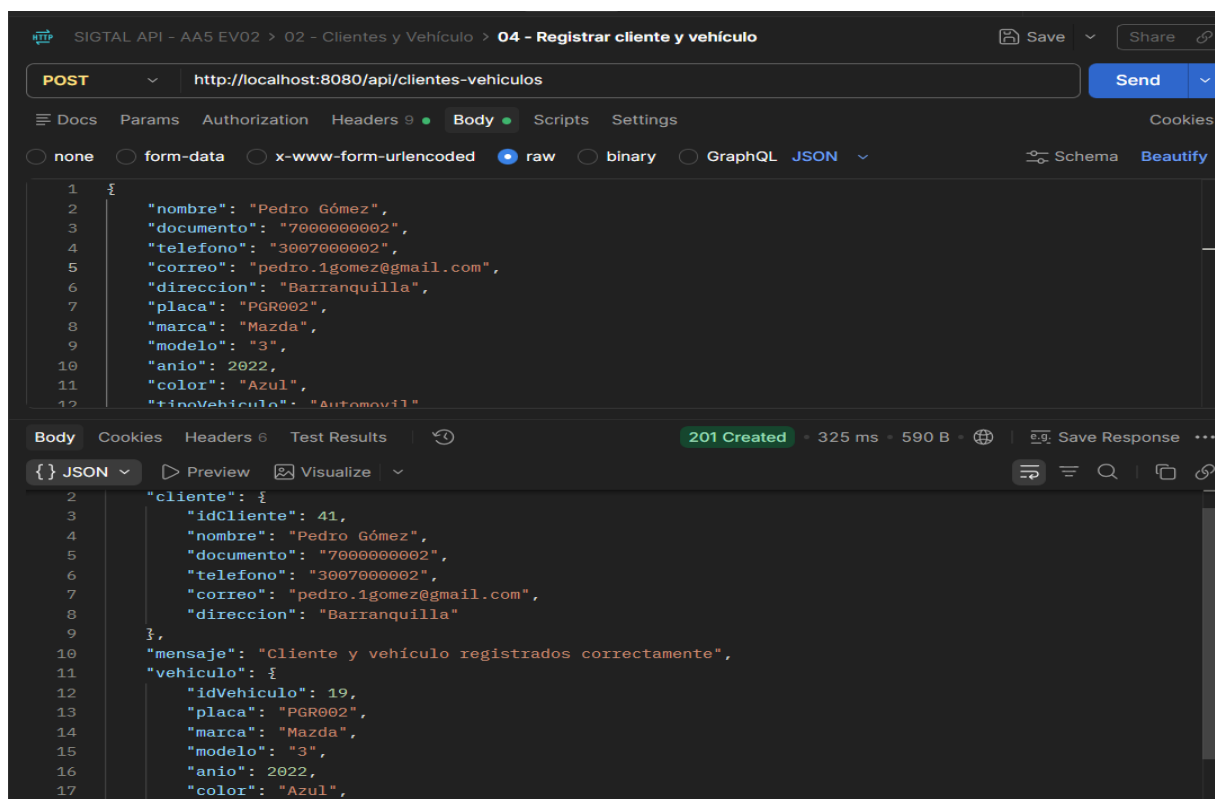
Prueba 3 - Vehículos por cliente

Se realizó la prueba 03 - Vehículos por cliente, verificando la consulta de los vehículos asociados a un cliente determinado.



Prueba 4 - Registrar cliente y vehículo

Se realizó la prueba 04 - Registrar cliente y vehículo, verificando el servicio utilizado para registrar la información correspondiente al cliente y su vehículo.





Prueba 5 - Documento duplicado

Se realizó la prueba 05 - Documento duplicado, con el objetivo de comprobar que el sistema controle el registro de clientes que utilizan un documento que ya se encuentra registrado.

The screenshot shows a REST client interface for a POST request to `http://localhost:8080/api/clientes-vehiculos`. The request body is a JSON object representing a client with a duplicate document. The response is a `400 Bad Request` with the message: `"error": "Ya existe un cliente con el documento: 7000000001"`.

```
1 {
2   "nombre": "Cliente Documento Duplicado",
3   "documento": "7000000001",
4   "telefono": "3007000002",
5   "correo": "duplicado.documento@gmail.com",
6   "direccion": "Barranquilla",
7   "placa": "DCC002",
8   "marca": "Toyota",
9   "modelo": "Corolla",
10  "anio": 2021,
11  "color": "Negro",
12  "tipoVehiculo": "Automovil"
13 }
```

Body Cookies Headers 5 Test Results 400 Bad Request · 47 ms · 273 B · Save Response

+ Continue in Agent Mode Diagnose Decline

{ JSON Preview Debug with AI

```
1 {
2   "error": "Ya existe un cliente con el documento: 7000000001"
3 }
```

Prueba 6 - Placa duplicada

Se realizó la prueba 06 - Placa duplicada, verificando que la API controle el registro de vehículos cuya placa ya existe en el sistema.

The screenshot shows a REST client interface for a POST request to `http://localhost:8080/api/clientes-vehiculos`. The request body is a JSON object representing a client with a duplicate plate. The response is a `400 Bad Request` with the message: `"error": "Ya existe un vehiculo con la placa: PGR001"`.

```
1 {
2   "nombre": "Cliente Placa Duplicada",
3   "documento": "7000000003",
4   "telefono": "3007000003",
5   "correo": "duplicado.placa@gmail.com",
6   "direccion": "Barranquilla",
7   "placa": "PGR001",
8   "marca": "Chevrolet",
9   "modelo": "Onix",
10  "anio": 2023,
11  "color": "Rojo",
12  "tipoVehiculo": "Automovil"
13 }
```

Body Cookies Headers 5 Test Results 400 Bad Request · 59 ms · 267 B · Save Response

+ Resolve duplicate plate error Diagnose Decline

{ JSON Preview Debug with AI

```
1 {
2   "error": "Ya existe un vehiculo con la placa: PGR001"
3 }
```



Prueba 7-Actualizar cliente

Se realizó la prueba 07-Actualizar cliente, verificando el funcionamiento del servicio destinado a modificar la información de un cliente registrado.

The screenshot shows a REST client interface with the following details:

- Method:** PUT
- URL:** `http://localhost:8080/api/clientes-vehiculos/cliente/31`
- Body (raw JSON):**

```
1 {
2   "idCliente": 31,
3   "nombre": "Juan Carlos Pérez",
4   "documento": "987654321",
5   "telefono": "3015551234",
6   "correo": "juan.perez@gmail.com",
7   "direccion": "Calle 50 # 40-25"
8 }
```
- Response:** 200 OK, 144 ms, 398 B. The response body is a JSON object identical to the request body.

Prueba 8-Actualizar vehículo

Se realizó la prueba 08-Actualizar vehículo, comprobando que la información de un vehículo registrado pueda ser actualizada correctamente.

The screenshot shows a REST client interface with the following details:

- Method:** PUT
- URL:** `http://localhost:8080/api/clientes-vehiculos/vehiculo/19`
- Body (raw JSON):**

```
1 {
2   "placa": "PGR002",
3   "marca": "Mazda",
4   "modelo": "3 Touring",
5   "anio": 2023,
6   "color": "Gris",
7   "tipoVehiculo": "Automovil"
8 }
```
- Response:** 200 OK, 165 ms, 366 B. The response body is a JSON object identical to the request body.



Prueba 9-Eliminar vehículo

Se realizó la prueba 09-Eliminar vehículo, verificando el funcionamiento del proceso de eliminación de un vehículo registrado.

SIGTAL API - AA5 EV02 > 02 - Clientes y Vehículo > **09-Eliminar vehículo**

DELETE `http://localhost:8080/api/clientes-vehiculos/vehiculo/20` **Send**

Docs Params Authorization Headers 6 **Body** Scripts Settings Cookies

☒ none ☐ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL

This request does not have a body

Body Cookies Headers 6 Test Results **200 OK** • 56 ms • 288 B • Save Response

JSON Preview Visualize

```
1 {
2   "mensaje": "Vehículo eliminado correctamente"
3 }
```

Prueba 10-Verificar eliminación del vehículo

Se realizó la prueba 10-Verificar eliminación del vehículo, con el propósito de comprobar que el vehículo eliminado anteriormente ya no se encuentre disponible para consulta.

SIGTAL API - AA5 EV02 > 02 - Clientes y Vehículo > **10-Verificar eliminación del vehículo**

DELETE `http://localhost:8080/api/clientes-vehiculos/vehiculo/20` **Send**

Docs **Params** Authorization Headers 6 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 6 Test Results **404 Not Found** • 60 ms • 287 B • Save Response

Continue in Agent Mode **Diagnose** Decline

JSON Preview Debug with AI

```
1 {
2   "error": "Vehículo no encontrado con ID: 20"
3 }
```



Prueba 11-Eliminar cliente

Se realizó la prueba 11-Eliminar cliente, verificando el funcionamiento del servicio encargado de eliminar un cliente registrado.

SIGTAL API - AA5 EV02 > 02 - Clientes y Vehículo > **11-Eliminar cliente**

DELETE <http://localhost:8080/api/clientes-vehiculos/38> **Send**

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 6 Test Results **200 OK** · 127 ms · 304 B · Save Response

JSON Preview Visualize

```
1 {
2   "mensaje": "Cliente y sus vehículos eliminados correctamente"
3 }
```

Prueba 12-Consultar cliente eliminado

Se realizó la prueba 12-Consultar cliente eliminado, comprobando que el sistema identifique correctamente que el cliente eliminado ya no se encuentra disponible.

SIGTAL API - AA5 EV02 > 02 - Clientes y Vehículo > **12-Consultar cliente eliminado**

GET <http://localhost:8080/api/clientes-vehiculos/35> **Send**

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 6 Test Results **404 Not Found** · 42 ms · 285 B · Save Response

Investigate the 404 error **Diagnose** Decline

JSON Preview Debug with AI

```
1 {
2   "error": "Cliente no encontrado con ID: 35"
3 }
```



Pruebas de Inventario

Prueba 1-Listar productos

Se realizó la prueba 01-Listar productos, verificando que la API permita consultar los productos registrados en el módulo de inventario.

```
SIGTAL API - AA5 EV02 > 03 - Inventario > 01-Listar productos
GET http://localhost:8080/api/inventario
200 OK - 81 ms - 817 B
[{"idProducto": 6, "codigo": "PROD-006", "nombre": "Filtro de aire", "categoria": "Filtros", "marca": "Bosch", "descripcion": "", "cantidad": 33, "stockMinimo": 11, "precioCompra": 35000.00, "precioVenta": 45000.00, "proveedor": "Proveedor Automotriz", "estado": "ACTIVO", "fechaRegistro": "2026-09-20T12:24:41"}, {"idProducto": 7, "codigo": "PROD-007", "nombre": "Filtro de aire", "categoria": "BATERIAS", "marca": "Bosch", "descripcion": "", "cantidad": 38, "stockMinimo": 15, "precioCompra": 45000.00, "precioVenta": 50000.00, "proveedor": "Proveedor Automotriz", "estado": "ACTIVO", "fechaRegistro": "2026-09-20T12:28:11"}]
```

Prueba 2-Registrar producto

Se realizó la prueba 02-Registrar producto, verificando el funcionamiento del servicio utilizado para registrar un nuevo producto en el inventario.

```
SIGTAL API - AA5 EV02 > 03 - Inventario > 02-Registrar producto
POST http://localhost:8080/api/inventario
Body: {"codigo": "PROD-008", "nombre": "Aceite 10W40", "categoria": "Lubricantes", "marca": "Castrol", "descripcion": "Aceite para motor 10W40", "cantidad": 20, "stockMinimo": 5, "precioCompra": 28000, "precioVenta": 35000, "proveedor": "Proveedor Automotriz", "estado": "ACTIVO"}
201 Created - 223 ms - 555 B
{"idProducto": 8, "codigo": "PROD-008", "nombre": "Aceite 10W40", "categoria": "Lubricantes", "marca": "Castrol", "descripcion": "Aceite para motor 10W40", "cantidad": 20, "stockMinimo": 5, "precioCompra": 28000, "precioVenta": 35000, "proveedor": "Proveedor Automotriz", "estado": "ACTIVO", "fechaRegistro": "2026-10-02T11:47:43.4389453"}
```



Prueba 3-Consultar producto por id

Se realizó la prueba 03-Consultar producto por id, comprobando que el sistema permita consultar un producto específico utilizando su identificador.

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://localhost:8080/api/inventario/8
- Status:** 200 OK
- Response Body (JSON):**

```
{  "idProducto": 8,  "codigo": "PROD-008",  "nombre": "Aceite 10W40",  "categoria": "Lubricantes",  "marca": "Castrol",  "descripcion": "Aceite para motor 10W40",  "cantidad": 20,  "stockMinimo": 5,  "precioCompra": 28000.00,  "precioVenta": 35000.00,  "proveedor": "Proveedor Automotriz",  "estado": "ACTIVO",  "fechaRegistro": "2026-10-02T11:47:43"}
```

Prueba 4-Actualizar producto

Se realizó la prueba 04-Actualizar producto, verificando que la información de un producto existente pueda ser modificada.

The screenshot shows a REST client interface with the following details:

- Method:** PUT
- URL:** http://localhost:8080/api/inventario/8
- Status:** 200 OK
- Request Body (JSON):**

```
{  "codigo": "PROD-008",  "nombre": "Aceite 10W40 Premium",  "categoria": "Lubricantes",  "marca": "Castrol",  "descripcion": "Aceite sintético para motor",  "cantidad": 35,  "stockMinimo": 8,  "precioCompra": 30000,  "precioVenta": 39000,  "proveedor": "Proveedor Automotriz",  "estado": "ACTIVO"}
```
- Response Body (JSON):**

```
{  "idProducto": 8,  "codigo": "PROD-008",  "nombre": "Aceite 10W40 Premium",  "categoria": "Lubricantes",  "marca": "Castrol",  "descripcion": "Aceite sintético para motor",  "cantidad": 35,  "stockMinimo": 8,  "precioCompra": 30000,  "precioVenta": 39000,  "proveedor": "Proveedor Automotriz",  "estado": "ACTIVO",  "fechaRegistro": "2026-10-02T11:47:43"}
```



Prueba 5-Eliminar producto

Se realizó la prueba 05-Eliminar producto, comprobando el funcionamiento del proceso de eliminación de un producto del inventario.

The screenshot shows a REST client interface for a test named "05-Eliminar producto". The request is a DELETE to `http://localhost:8080/api/inventario/8`. The response is a 200 OK with a body of `{"mensaje": "Producto eliminado correctamente"}`.

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

```
1 {
2   "mensaje": "Producto eliminado correctamente"
3 }
```

Prueba 6-Verificar producto eliminado

Se realizó la prueba 06-Verificar producto eliminado, verificando que el producto eliminado anteriormente ya no pueda ser consultado.

The screenshot shows a REST client interface for a test named "06-Verificar producto eliminado". The request is a GET to `http://localhost:8080/api/inventario/8`. The response is a 404 Not Found with a body of `{"mensaje": "Producto no encontrado con ID: 8", "error": "Recurso no encontrado"}`.

```
1 {
2   "mensaje": "Producto no encontrado con ID: 8",
3   "error": "Recurso no encontrado"
4 }
```



Prueba 7-Código de producto duplicado

Se realizó la prueba 07-Código de producto duplicado, comprobando que el sistema controle el registro de productos que utilizan un código que ya se encuentra registrado.

```
POST http://localhost:8080/api/inventario

{
  "codigo": "PROD-006",
  "nombre": "Producto Duplicado",
  "categoria": "Prueba",
  "marca": "Prueba",
  "descripcion": "Prueba de código duplicado",
  "cantidad": 10,
  "stockMinimo": 2,
  "precioCompra": 10000,
  "precioVenta": 15000,
  "proveedor": "Proveedor Prueba",
  "estado": "ACTIVO"
}
```

```
{
  "mensaje": "Ya existe un producto con el código: PROD-006",
  "error": "Solicitud inválida"
}
```

Prueba 8-Actualizar producto

Se realizó la prueba 08-Actualizar producto, verificando nuevamente el funcionamiento de la actualización de la información de un producto existente.

```
PUT http://localhost:8080/api/inventario/9

{
  "codigo": "PROD-008",
  "nombre": "Filtro de aceite actualizado",
  "categoria": "Filtros",
  "marca": "Bosch",
  "descripcion": "Producto actualizado mediante API",
  "cantidad": 25,
  "stockMinimo": 5,
  "precioCompra": 30000,
  "precioVenta": 42000,
  "proveedor": "Proveedor Automotriz",
  "estado": "ACTIVO"
}
```

```
{
  "idProducto": 9,
  "codigo": "PROD-008",
  "nombre": "Filtro de aceite actualizado",
  "categoria": "Filtros",
  "marca": "Bosch",
  "descripcion": "Producto actualizado mediante API",
  "cantidad": 25,
  "stockMinimo": 5,
  "precioCompra": 30000,
  "precioVenta": 42000,
  "proveedor": "Proveedor Automotriz",
  "estado": "ACTIVO",
  "fechaRegistro": "2026-10-02T12:47:06"
}
```



Prueba 9-Producto inexistente

Se realizó la prueba 09-Producto inexistente, comprobando que la API identifica correctamente cuando se consulta un producto que no existe. El sistema responde con código HTTP 404 y proporciona un mensaje indicando que el producto no fue encontrado.

SIGTAL API - AA5 EV02 > 03 - Inventario > **09-Producto inexistente**

GET `http://localhost:8080/api/inventario/9` Send

Docs Params Authorization Headers 8 **Body** Scripts Settings Cookies

none form-data x-www-form-urlencoded **raw** binary GraphQL JSON Schema Beautify

```
1 {
2   "codigo": "PROD-999",
3   "nombre": "Producto inexistente",
4   "categoria": "Prueba",
5   "marca": "Prueba",
6   "descripcion": "Prueba de actualización con ID inexistente",
7   "cantidad": 10,
8   "stockMinimo": 2,
9   "precioCompra": 10000,
10  "precioVenta": 15000,
11  "proveedor": "Proveedor Prueba",
12  "estado": "ACTIVO"
13 }
```

Body Cookies Headers 6 Test Results 404 Not Found • 58 ms • 319 B • Save Response Diagnose Decline

Fix non-e

{ } JSON Preview Debug with AI

```
1 {
2   "mensaje": "Producto no encontrado con ID: 9",
3   "error": "Recurso no encontrado"
4 }
```

Prueba 10-Eliminar el producto ID 9

Se realizó la prueba 10-Eliminar el producto ID 9, verificando el proceso de eliminación del producto identificado con el ID 9.

SIGTAL API - AA5 EV02 > 03 - Inventario > **10-Eliminar el producto ID 9**

DELETE `http://localhost:8080/api/inventario/9` Send

Docs Params Authorization Headers 6 **Body** Scripts Settings Cookies

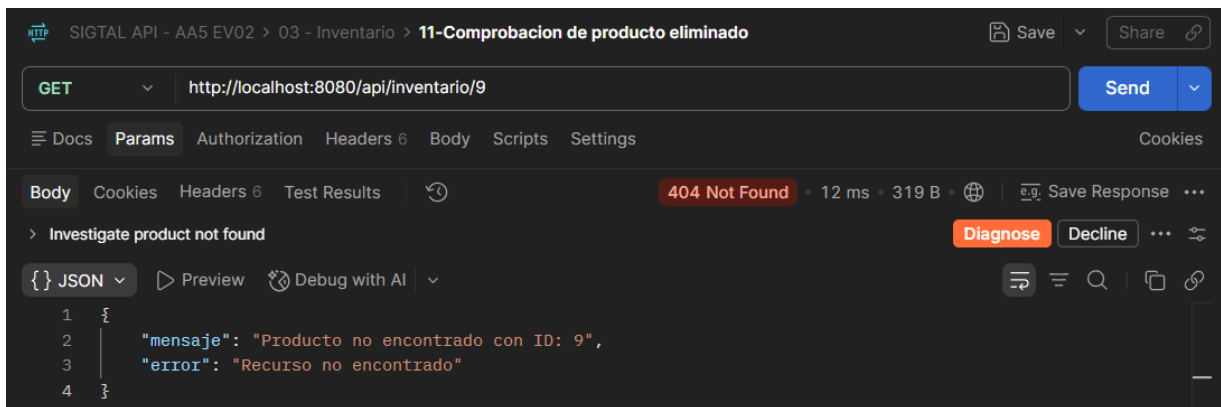
Body Cookies Headers 6 Test Results 200 OK • 573 ms • 287 B • Save Response

{ } JSON Preview Visualize

```
1 {
2   "mensaje": "Producto eliminado correctamente"
3 }
```

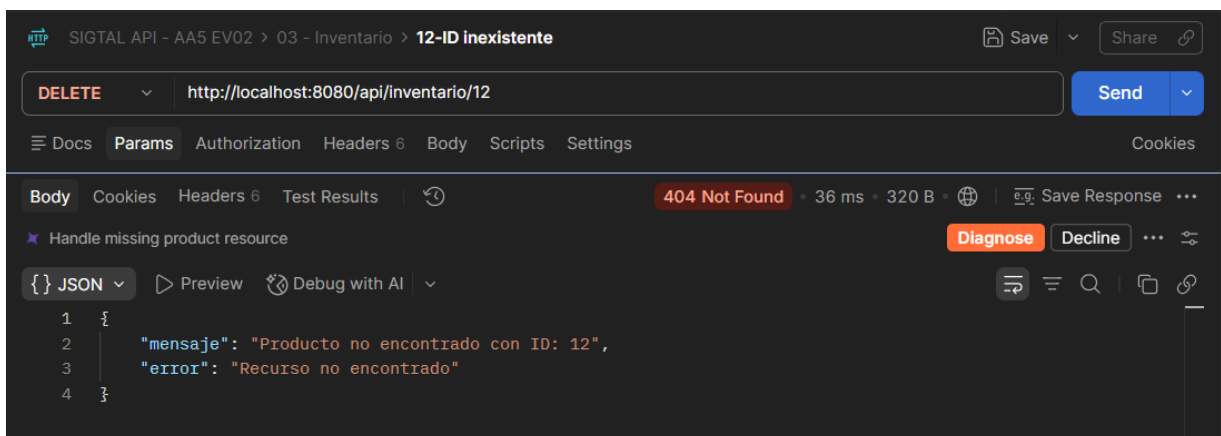
Prueba 11-Comprobación de producto eliminado

Se realizó la prueba 11-Comprobación de producto eliminado, con el objetivo de confirmar que el producto eliminado en la prueba anterior ya no se encuentre disponible en el sistema.



Prueba 12-ID inexistente

Se realizó la prueba 12-ID inexistente, verificando el comportamiento de la API cuando se solicita información utilizando un identificador que no se encuentra registrado.



8. Análisis de resultados

Los resultados obtenidos muestran que los servicios implementados responden de acuerdo con lo esperado para las operaciones verificadas.

El módulo Autenticación permite validar el inicio de sesión y controlar situaciones como usuarios inexistentes y contraseñas incorrectas.

El módulo Clientes y Vehículo permite gestionar la información de los clientes y sus vehículos mediante operaciones de consulta, registro, actualización y eliminación, además de realizar validaciones para evitar documentos y placas duplicadas.



Por su parte, el módulo Inventario permite gestionar productos mediante consulta, registro, actualización y eliminación, además de validar códigos de productos duplicados y controlar consultas de productos inexistentes.

En conjunto, las pruebas realizadas mediante Postman permiten comprobar el funcionamiento de los servicios implementados para los módulos incluidos en la API SIGTAL.

También se verificó el manejo de recursos inexistentes. La API retorna HTTP 404 y un mensaje descriptivo cuando el producto consultado no existe, permitiendo diferenciar una solicitud válida de una solicitud sobre un recurso inexistente.

Durante la ejecución se corrigieron problemas de compilación y manejo de excepciones. Finalmente, la aplicación inició correctamente en el puerto 8080 y respondió solicitudes desde Postman.

9. Conclusiones

- Las pruebas permitieron verificar los servicios REST implementados en la API **SIGTAL**.
- El módulo **Autenticación** fue validado mediante pruebas de inicio de sesión, usuario inexistente y contraseña incorrecta.
- El módulo **Clientes y Vehículo** fue validado mediante operaciones de consulta, registro, actualización y eliminación, incluyendo validaciones de documentos y placas duplicadas.
- El módulo **Inventario** fue validado mediante operaciones de consulta, registro, actualización y eliminación.
- Se comprobó el manejo de **recursos inexistentes y registros duplicados** en los módulos incluidos en las pruebas.
- La API funciona localmente en el **puerto 8080** y puede ser consumida mediante solicitudes HTTP utilizando Postman.
- Los resultados obtenidos constituyen evidencia del funcionamiento de los **endpoints incluidos en el alcance de la API SIGTAL**.
- Las pruebas realizadas permitieron comprobar que los servicios responden de acuerdo con los escenarios definidos para **Autenticación, Clientes y Vehículo e Inventario**.



11. Referencias

- Postman. (s. f.). Postman documentation. [Get started in Postman | Postman Docs](#)
- Spring. (s. f.). Spring Boot documentation. [Spring Boot :: Spring Boot](#)
- Spring. (s. f.). Spring Data JPA documentation. [Spring Data JPA :: Spring Data JPA](#)
- Oracle. (s. f.). Java documentation. [Documentación de Java - Comienza](#)